

Manipulação de Strings

Guia de referência rápida · Python 3

1. Criação de strings

Forma	Exemplo	Resultado
Aspas simples	<code>s = 'olá'</code>	<code>olá</code>
Aspas duplas	<code>s = "olá"</code>	<code>olá</code>
Multiline	<code>s = """linha1\nlinha2"""</code>	<code>linha1\nlinha2</code>
Raw string	<code>s = r'C:\\Users'</code>	<code>C:\\Users</code>
f-string	<code>nome="Ana"; f"Oi {nome}"</code>	<code>Oi Ana</code>
Repetição	<code>"ha" * 3</code>	<code>hahaha</code>
Concatenação	<code>"oi" + " " + "tudo"</code>	<code>oi tudo</code>

2. Fatiamento e indexação (s = 'Python')

Expressão	Descrição	Resultado
<code>s[0]</code>	primeiro char	<code>P</code>
<code>s[-1]</code>	último char	<code>n</code>
<code>s[1:4]</code>	índice 1 até 3	<code>yth</code>
<code>s[:3]</code>	do início até 2	<code>Pyt</code>
<code>s[3:]</code>	do índice 3 ao fim	<code>hon</code>
<code>s[-3:]</code>	últimos 3 chars	<code>hon</code>
<code>s[::2]</code>	de 2 em 2	<code>Pto</code>
<code>s[::-1]</code>	invertido	<code>nohtyP</code>
<code>len(s)</code>	comprimento	<code>6</code>

3. Transformação de caso (s = 'olá mundo')

Método	Resultado
<code>s.upper()</code>	<code>OLÁ MUNDO</code>
<code>s.lower()</code>	<code>olá mundo</code>
<code>s.capitalize()</code>	<code>Olá mundo</code>

Método	Resultado
<code>s.title()</code>	Olá Mundo
<code>s.swapcase()</code>	OLÁ MUNDO → olá mundo

4. Busca e verificação (s = 'banana')

Método / Operador	Resultado
<code>"ana" in s</code>	True
<code>s.find('ana')</code>	1 (primeiro índice)
<code>s.rfind('ana')</code>	3 (último índice)
<code>s.index('nan')</code>	2 (erro se não achar)
<code>s.count('a')</code>	3
<code>s.startswith('ban')</code>	True
<code>s.endswith('na')</code>	True

5. Substituição e limpeza

Método	Resultado	
<code>replace(old, new)</code>	<code>"oi oi".replace("oi", "tchau")</code>	tchau tchau
<code>strip()</code>	<code>" oi ".strip()</code>	oi
<code>lstrip()</code>	<code>" oi ".lstrip()</code>	oi
<code>rstrip()</code>	<code>" oi ".rstrip()</code>	oi
<code>strip(chars)</code>	<code>####ok####.strip("#")</code>	ok
<code>removeprefix()</code>	<code>"Mr. João".removeprefix("Mr. ")</code>	João
<code>removesuffix()</code>	<code>"arquivo.py".removesuffix(".py")</code>	arquivo

6. Split e Join

Método	Exemplo	Resultado
<code>split()</code>	<code>"a b c".split()</code>	['a', 'b', 'c']
<code>split(sep)</code>	<code>"a,b,c".split(",")</code>	['a', 'b', 'c']
<code>split(sep, n)</code>	<code>"a,b,c".split(",", 1)</code>	['a', 'b,c']
<code>rsplit(sep, n)</code>	<code>"a,b,c".rsplit(",", 1)</code>	['a,b', 'c']
<code>splitlines()</code>	<code>"a\nb\nc".splitlines()</code>	['a', 'b', 'c']
<code>join(iter)</code>	<code>", ".join(["x","y","z"])</code>	x, y, z

Método	Exemplo	Resultado
partition(sep)	"a:b:c".partition(":")	('a', ':', 'b:c')

7. Alinhamento e formatação

Método	Exemplo	Resultado
center(n, ch)	"oi".center(10, "-")	----oi----
ljust(n, ch)	"oi".ljust(8, ".")	oi.....
rjust(n, ch)	"oi".rjust(8, ".")	oi
zfill(n)	"42".zfill(6)	000042
f'{v:.2f}'	f'{3.14159:.2f}'	3.14
f'{v:,}'	f'{1000000:,}'	1,000,000
f'{v:>10}'	f'{"oi":>10}'	oi
f'{v:<10}'	f'{"oi":<10}'	oi
f'{v:^10}'	f'{"oi":^10}'	oi

8. Métodos de verificação (is...)

Método	Retorna True quando...
s.isalpha()	todos os chars são letras
s.isdigit()	todos os chars são dígitos
s.isnumeric()	chars numéricos (inclui ² , ½...)
s.isalnum()	letras e/ou dígitos, sem espaço
s.isspace()	apenas espaços/tabs/newlines
s.isupper()	todas as letras são maiúsculas
s.islower()	todas as letras são minúsculas
s.istitle()	formato de título (Title Case)
s.isprintable()	todos os chars são imprimíveis

9. Encode / Decode

Operação	Exemplo	Resultado
encode()	"olá".encode("utf-8")	b'ol\xc3\xa1'
decode()	b'ol\xc3\xa1'.decode("utf-8")	olá
ord(ch)	ord("A")	65

Operação	Exemplo	Resultado
<code>chr(n)</code>	<code>chr(65)</code>	A

10. Dicas essenciais

Strings são imutáveis — todo método retorna uma nova string, nunca altera a original.

f-strings são o jeito moderno de formatar. Prefira `f"Olá {nome}"` a `"%s" % nome`.

Concatenação em loop é lenta. Use `"".join(lista)` ao construir strings grandes.

raw strings (`r"...`) ignoram `\n`, `\t` etc. Úteis em regex e caminhos Windows.